Rendszerterv

Web:Learning

Fazekas Zsolt
2025. november 11.

1. Bevezetés

2019-ben kibontakozó COVID-19 járvány miatt jelenléti oktatásra nem volt lehetőség, így számos alternatívát kellett alkalmazni a tananyagok leadására, így egyre népszerűbb lett az online, jelenlétet nem igénylő kurzusok, mint ahogy a felső oktatási intézményekben a távoktatás, a levelező tagozatok. 2025-ben, 6 évvel a járvány után, annyira bele épült a hétköznapban, hogy számos intézmény a tananyagait is feltölti a tanulóknak, hogy bármikor elérhessék, kiegészítsék az órán elhangzott információkat vagy jobban fel tudjanak készülni a számonkérésekre.

1.1. HTML - Hyper Text Markup Language

A **HTML** (HyperText Markup Language) a webes felületek szerkezetének leírására szolgáló szabványos leírónyelv. Nem programozási nyelv, hanem strukturális markup, amely hierarchikus módon írja le a dokumentum elemeit. A HTML dokumentum **DOM (Document Object Model)** struktúrává alakul, amelyet a böngészők renderelnek. A HTML deklaratív módon határozza meg a tartalmi elemeket, mint például szövegek, képek, űrlapok, táblázatok, hivatkozások vagy multimédiás objektumok.
A modern HTML, különösen a **HTML5**, számos szemantikus elemet vezetett be (**<header>**, **<section>**, **<article>**, **<footer>**, stb.), amelyek elősegítik a dokumentum logikai tagolását, továbbá javítják az akadálymentesítést és a keresőoptimalizálást (SEO). A HTML5 rendelkezik beépített API-kal is, például **Canvas API**, **Audio/Video API**, és **LocalStorage**, amelyek lehetővé teszik a webalkalmazások fejlettebb funkcionális működését külső plugin nélkül.

1.2. CSS - Cascading Style Sheets

A **CSS** (Cascading Style Sheets) a weboldalak megjelenésének és vizuális formázásának definiálására használatos stílusleíró nyelv. Szerepe a HTML tartalomtól való teljes elkülönítés: míg a HTML a szerkezetet, a CSS a prezentációt határozza meg. A CSS kaskád és öröklődés alapú működése lehetővé teszi a stílusdefiníciók hierarchikus alkalmazását, ezzel támogatva a moduláris és karbantartható dizájn felépítését.
A modern webfejlesztésben elterjedtek a fejlettebb CSS funkciók és technológiák, mint például:

- **Flexbox** és **CSS Grid**: reszponzív és rugalmas layout rendszerek
- **Media Queries**: eszközfüggő adaptív megjelenítés
- **CSS változók** (custom properties): újrafelhasználható design tokenek
- **Preprocesszorok** (SASS, SCSS): változók, mixinek, függvények használata
- **BEM módszertan**: komponensalapú névadási konvenció

A CSS-t a böngésző renderelési folyamata értelmezi, amely során a HTML struktúrához stílusszabályokat kapcsol, majd létrehozza a render tree-t, végül megjeleníti azt a képernyőn.

1.3. JavaScript

A **JavaScript** egy dinamikus, prototípus alapú, **többparadigmájú programozási nyelv**, amely elsődlegesen a kliens oldali interaktivitást és logikai működést biztosítja a weboldalakon. A JavaScriptet a böngészők beépített futtatókörnyezete (pl. **V8**, **SpiderMonkey**, **Chakra**) hajtja végre, amely lehetővé teszi a DOM manipulációt, eseménykezelést, aszinkron működést és hálózati kommunikációt.

A nyelv támogatja az **event-driven**, az **OOP**, valamint a **funkcionális programozás** elemeit is. ES6+ szabványoknak köszönhetően olyan nyelvi elemekkel bővült, mint:

- **modulos rendszer** (import, export)
- **arrow-függvények**
- **async/await** és **Promise** alapú aszinkronitás
- **osztály** alapú szintaxis (class)
- **destrukturálás**, **spread operátor**
- **Map**, **Set**, **WeakMap** adattípusok

A JavaScriptből kinőtt ökoszisztéma hatalmas:

Node.js segítségével szerveroldalon, **Angular** / **React** / **Vue** segítségével pedig **SPA** alkalmazásokban is meghatározó szereplő. A nyelv képezi a modern webalkalmazások valós idejű működésének alapját, például **WebSocketek**, **Fetch API**, **Service Worker** és más **böngésző API**-k révén.

2. Az Alkalmazás

A rendszer két fő felhasználói szerepkörre épül: Oktató és Hallgató. Bármely regisztrált felhasználó felveheti az Oktató szerepkört, és létrehozhat egy vagy több kurzust. Az Oktató feltölthet különböző típusú tananyagokat (pl. PDF dokumentumok, videó linkek, szöveges leírások. A rendszer kulcsfontosságú eleme az egyszerű hozzáférés-kezelés: minden kurzushoz generálódik egy egyedi meghívó kód. Az Oktató ezt a kódot megosztva hívhat meg Hallgatókat a kurzusára. A Hallgatók a kód birtokában csatlakozhatnak a kurzushoz, megtekinthetik a tananyagokat, és nyomon követhetik a saját haladásukat.

Decentralizáltság: Nincs központi adminisztráció; bárki lehet oktató és létrehozhat tartalmat.

Egyszerűség: A felhasználói felület (UI) és a funkciók az intuitív használatra fókuszálnak, elkerülve a komplex rendszerek túlzsúfoltságát.

Rugalmasság: Ideális kis létszámú képzések, workshopok, egyetemi szemináriumok vagy céges onboarding anyagok kezelésére.

Modern technológia: A választott technológiai stack biztosítja a rendszer skálázhatóságát, biztonságát és karbantarthatóságát.

2.1. Backend (Laravel)

Laravel (PHP keretrendszer) választása mellett szólt a gyors fejlesztési ciklus (Rapid Application Development), a beépített biztonsági funkciók (pl. ORM, authentikáció), valamint a letisztult, elegáns szintaxis. A Laravel biztosítja a RESTful API végpontokat, amelyek kiszolgálják a kliensoldali alkalmazást.

Alkalmas biztonságos, állapotmentes (stateless) RESTful API-k gyors építésére. A beépített Sanctum vagy Passport csomagok egyszerűvé teszik az Angular kliens authentikációját (pl. token alapú azonosítással).

Eloquent ORM: A Laravel beépített Object-Relational Mapper-e (Eloquent) absztraktálja az adatbázis-műveleteket. Ez lehetővé teszi, hogy tiszta, olvasható PHP szintaxissal kezeljük az adatbázis-rekordokat (pl. User::find(1)->courses()), ahelyett, hogy nyers SQL lekérdezéseket írnánk.

Fejlesztői élmény: A Laravel "elegáns" szintaxisa, részletes dokumentációja és hatalmas ökoszisztémája (pl. beépített migrációk, seeder-ek) jelentősen felgyorsítja a fejlesztési és karbantartási folyamatokat.

Biztonság: A keretrendszer alapértelmezetten védelmet nyújt a leggyakoribb sebezhetőségek ellen (pl. SQL injection az Eloquent révén, Cross-Site Scripting (XSS) a Blade template-ekben).

2.2. Frontend (Angular)

A kliensoldali megjelenítésért és interakciókért egy Angular (TypeScript alapú keretrendszer) felel. Az Angular komponensalapú architektúrája tiszta és jól strukturált kódot eredményez. Képes egy reszponzív, gyors és dinamikus egyoldalas alkalmazás (Single Page Application - SPA) létrehozására, amely kiváló felhasználói élményt nyújt.

TypeScript nyelvre épül, ami a JavaScript erősen típusos kiterjesztése. Ez már fordítási időben kiszűri a hibák jelentős részét, és sokkal robusztusabb, könnyebben karbantartható kódot eredményez, ami egy nagy méretű projektnél kulcsfontosságú.

Komponensalapú Architektúra: A rendszer jól elkülönített, újrafelhasználható komponensekből (pl CourseListComponent, LearningMaterialComponent) épül fel. Ez a felépítés tiszta és logikus struktúrát kényszerít ki.

Komplett keretrendszer (Opinionated): Az Angular nem csak egy "könyvtár" (mint a React), hanem egy teljes "keretrendszer". Mindent biztosít, amire szükség van: beépített routert a navigációhoz, HTTP klienst az API-hívásokhoz, és hatékony űrlapkezelést. Ez egységes kódbázist eredményez.

Teljes felelősség-szétválasztás (SoC): Az Angular alkalmazás (SPA) teljesen különálló életet él a Laravel backendtől. Ez a tiszta szétválasztás lehetővé teszi, hogy a két réteg párhuzamosan fejleszthető és tesztelhető legyen.

2.3. Adatbázis (SQLite)

A rendszer adatainak tartós tárolására egy relációs, de szerver nélküli (**serverless**) adatbázis-kezelő, az SQLite mellett döntöttem.

Egyszerűség és "Zero-Configuration": A MySQL-lel vagy PostgreSQL-lel ellentétben az SQLite nem igényel külön adatbázis-szerver telepítést, futtatást vagy adminisztrációt. Az egész adatbázis egyetlen, hordozható fájlban (pl. database/database.sqlite) található a projekt mappájában.

Kiváló hordozhatóság (Portability):}Mivel az adatbázis a projekt része, a fejlesztői környezet beállítása (akár egy új gépen, akár a konzulens vagy bíráló gépén) triviális. Nincs szükség adatbázis-felhasználók vagy jelszavak kezelésére, ami jelentősen gyorsítja a fejlesztést és a tesztelést.

Tökéletes Laravel Integráció: A Laravel Eloquent ORM teljes mértékben támogatja az SQLite-ot. A relációs adatszerkezet (Felhasználók, Kurzusok kapcsolatai) ugyanúgy modellezhető és kezelhető, mintha egy nagy szerver-alapú adatbázist használnánk.

A projekt igényeinek megfelelő: Egy szakdolgozat-léptékű alkalmazásnál, ahol a felhasználói bázis és az egyidejű írási műveletek száma mérsékelt, az SQLite teljesítménye bőségesen elegendő. Relációs képességei (pl. idegen kulcsok támogatása) biztosítják az adatkonzisztenciát.

Az adatbázis logikai felépítése (Entitás-Kapcsolati Diagram, ERD) a dbdiagram.io online eszközzel készült el, amely vizuálisan ábrázolja fejezetben definiált táblákat és azok kapcsolatait.